

A Cloud-Native Gateway Architecture for Cost-Efficient Large Language Model Inference Serving

Anonymous Author(s) *Department of Computer Science and Engineering*
University / Research Institute
 City, Country
 email@example.com

Abstract—Deploying large language models (LLMs) in production requires more than a model serving engine. Teams must layer authentication, rate limiting, traffic shaping, observability, and graceful degradation on top of the raw inference API—and they often build these concerns from scratch for each deployment. This paper presents the design, implementation, and evaluation of a cloud-native inference gateway that separates platform concerns from model serving in a reusable, Kubernetes-native architecture. The gateway exposes an OpenAI-compatible chat completions endpoint, enforces per-key rate limiting with a sliding-window algorithm, and exports Prometheus metrics for request volume, upstream status distribution, and end-to-end latency. We evaluate the system under three deployment modes: a CPU-based mock backend for fast iteration, a GPU-accelerated real vLLM backend, and a hybrid topology where the gateway runs inside Kubernetes while the model server lives on a dedicated GPU host. Benchmark results show that the gateway adds less than 5 ms of median overhead relative to direct model serving, while rate limiting effectively caps per-tenant consumption under a 60-request-per-minute policy with zero false positives. The full system—gateway, mock backend, monitoring stack, and Kubernetes manifests—is released as open-source software. We discuss the engineering trade-offs we encountered, including the tension between in-memory rate-limit state and horizontal scaling, the practical limits of the sidecar observability pattern, and the challenges of maintaining a single codebase across mock, real, and hybrid deployment paths.

Index Terms—LLM inference, cloud-native, API gateway, rate limiting, Kubernetes, Prometheus, observability

I. INTRODUCTION

The past three years have seen large language models move from research artifacts to production infrastructure at an unprecedented pace. Open-weight models such as Llama [1], Qwen [2], and DeepSeek [3] now deliver quality that rivals closed APIs, and serving engines like vLLM [4] and TensorRT-LLM have made it feasible to run these models on modest GPU clusters. What remains under-addressed, however, is the *operational scaffolding* that turns a model server into a reliable, observable, multi-tenant production service.

Consider a concrete scenario. A platform team deploys vLLM behind a Kubernetes service. The model responds to HTTP requests on port 8000. The team now needs to (a) authenticate callers, (b) ensure no single client saturates the model’s limited context window or generation budget, (c) monitor request throughput and error rates, (d) fail gracefully when

the model server is overloaded or unreachable, and (e) support multiple deployment topologies—local development, CI/CD pipelines, staging clusters without GPUs, and production GPU nodes. None of these concerns are addressed by the model serving engine itself. Each team that deploys an LLM ends up re-implementing roughly the same middleware layer, often in an ad-hoc manner that couples tightly to a specific model or infrastructure.

This paper describes a cloud-native inference gateway that tackles these concerns head-on. The gateway is a FastAPI application deployed as a Kubernetes Deployment, positioned between any OpenAI-compatible client and any OpenAI-compatible model backend. Its core responsibilities are authentication via Bearer tokens, per-key sliding-window rate limiting, upstream request forwarding with configurable timeouts, and Prometheus-compatible metrics export. The design follows the *separation of concerns* principle: model serving concerns (batching, memory management, token sampling) live inside the serving engine, while platform concerns (auth, rate limiting, routing, observability) live inside the gateway.

We make the following contributions:

- 1) **A reusable gateway architecture** for LLM inference that cleanly decouples platform middleware from model serving. The architecture supports three deployment modes—mock, real GPU, and hybrid—from a single codebase.
- 2) **A lightweight sliding-window rate limiter** with per-API-key granularity, implemented in under 40 lines of Python using only standard-library data structures, that adds negligible latency under load.
- 3) **A comprehensive evaluation** covering baseline latency, rate-limit enforcement, and real-model throughput under the hybrid deployment topology, using k6 as the load generator and a Prometheus/Grafana observability stack.
- 4) **An open-source release** of the complete platform, including Kubernetes manifests for Gateway, mock backend, real vLLM, Prometheus, and Grafana, plus PowerShell automation scripts and k6 benchmark definitions.

The remainder of the paper is organized as follows. Section II surveys the landscape of LLM serving and API gateway design. Section III details the gateway architecture and each middleware component. Section IV describes the three

deployment topologies and their Kubernetes implementation. Section V presents our experimental evaluation. Section VI reflects on engineering trade-offs and lessons learned. Section VII positions our work within the broader literature. Section IX concludes and outlines directions for future work.

II. BACKGROUND AND MOTIVATION

A. The LLM Inference Serving Landscape

Modern LLM inference serving is dominated by a handful of open-source engines. vLLM [4] introduced PagedAttention, a memory management technique that nearly eliminates KV-cache fragmentation, enabling much higher throughput than the naive HuggingFace Transformers pipeline. Text Generation Inference (TGI) by HuggingFace provides a similar feature set with an emphasis on production readiness and watermarking. TensorRT-LLM by NVIDIA delivers the highest raw throughput on NVIDIA hardware through aggressive kernel fusion and quantization, at the cost of a more complex build-and-compile workflow. More recently, SGLang [5] and LightLLM have explored radix-aware prefix caching and structured generation.

What all these engines have in common is that they expose an OpenAI-compatible HTTP API—typically `POST /v1/chat/completions` with a JSON payload specifying the model, messages, and generation parameters. This standardization is a genuine step forward for interoperability. But the engines themselves are designed as single-tenant, single-model servers. They provide no built-in authentication, no per-user quotas, no request routing beyond a single upstream endpoint, and no observability beyond raw access logs. From a platform engineering perspective, they are building blocks, not turnkey services.

The gap between what a serving engine provides and what a production service requires has been partially filled by commercial offerings. OpenAI’s API, Anthropic’s API, and cloud-provider managed services like Amazon Bedrock all wrap their inference engines in substantial middleware: API key management, usage-based billing, rate limiting, content filtering, and detailed analytics dashboards. But for teams running open-weight models on their own infrastructure—whether for cost, latency, data residency, or customization reasons—no equivalent open-source middleware layer exists. Projects like LiteLLM and OpenRouter address some of these concerns at the client-library level, but they do not provide a server-side gateway that can be deployed in front of arbitrary model backends.

B. API Gateway Patterns in Microservice Architectures

The API gateway pattern is well-established in microservice architectures [6]. Gateways such as NGINX, Envoy, Kong, and Traefik handle cross-cutting concerns—TLS termination, authentication, rate limiting, request routing, and logging—so that backend services can focus on business logic. In a typical web application, the gateway routes requests to dozens or hundreds of microservices based on URL path, headers, or service discovery.

The LLM inference setting, however, introduces constraints that make off-the-shelf API gateways a poor fit. First, the chat

completions endpoint carries a JSON payload that the gateway may need to inspect—for example, to extract or default the `model` field for routing decisions. Most traditional gateways operate at the HTTP layer and do not parse request bodies. Second, LLM requests are long-lived: a single chat completion can take tens of seconds for a long response, and streaming responses via Server-Sent Events (SSE) require the gateway to understand and relay chunked transfer encoding. Third, the operational metrics that matter for LLM inference—tokens generated, time-to-first-token (TTFT), inter-token latency—are model-specific and cannot be captured by generic HTTP-level metrics alone, though HTTP-level metrics remain valuable as a first approximation.

C. Why Not Just Use Envoy or Kong?

We considered deploying an existing gateway and rejected it for three practical reasons. One: the configuration surface for Kong or Envoy is substantial, and maintaining an equivalent set of L7 routing rules, rate-limit policies, and auth plugins across YAML files would, in our judgment, be more complex than a domain-specific Python gateway of roughly 150 lines. Two: these gateways are not designed to parse or validate the OpenAI chat completion payload format, which means they cannot default a missing `model` field or reject malformed messages before they reach the model server. Three: the Prometheus metrics we wanted—counters keyed by model name and status code, histograms keyed by HTTP method and path—would require custom Lua filters or WebAssembly plugins in Envoy, which again adds complexity without commensurate benefit for a single-purpose inference gateway.

To be clear, we are not arguing against general-purpose gateways. For organizations that already operate an Envoy or Kong mesh, extending that mesh with LLM-aware filters is a reasonable path. Our design is aimed at teams that want a lightweight, self-contained inference gateway that they can understand, modify, and deploy in an afternoon, without taking on the operational burden of a full API management platform.

D. Design Goals

From the above analysis, we distilled four design goals that guided our implementation:

- 1) **OpenAI compatibility.** The gateway must accept and forward the standard `/v1/chat/completions` request format so that any OpenAI-compatible client—including the Python `openai` library, LangChain, and llama.cpp—can connect without modification.
- 2) **Multi-mode deployment.** A single gateway binary should support at least three backend configurations: a CPU-only mock server for testing and CI, a real GPU-accelerated vLLM server for production, and a hybrid topology where the gateway lives inside Kubernetes but the model server runs on a separate GPU host.
- 3) **Minimal operational overhead.** The gateway should add negligible latency (target: under 10 ms median overhead) and should degrade gracefully when the upstream model server is unreachable.

- 4) **Observability-first design.** Every request should produce Prometheus-compatible metrics covering Gateway-level latency, upstream status distribution by model, and HTTP status code distribution by path, without requiring additional instrumentation in the model server.

E. The Self-Hosting Imperative

Before diving into the design, it is worth articulating why self-hosting LLMs has become such an important deployment pattern, and why the middleware problem matters for that pattern. The surface-level answer is cost: at the time of writing, serving a 7B-parameter model on a single A100 GPU costs roughly \$1.20–\$1.50 per hour in cloud GPU instances, while the equivalent API call volume through a commercial provider can easily exceed \$10–\$20 per hour for sustained workloads. For an always-on chatbot or an internal code-assistance tool that serves hundreds of developers, the self-hosted approach often pays for the GPU investment within weeks.

But cost is only part of the story. Three other factors drive self-hosting adoption among the teams we have spoken with. Data residency: organizations in regulated industries (finance, healthcare, government) often cannot send user queries to external API providers, regardless of the provider’s compliance certifications. Customization: fine-tuned models—whether instruction-tuned with proprietary documentation, RLHF-calibrated for a specific domain’s safety requirements, or merged from multiple open-weight checkpoints—cannot run on most managed API platforms. Latency: for real-time applications like code completion or conversational search, the additional network hop to a cloud API provider adds 50–200 ms of round-trip time that is entirely avoidable by co-locating the inference service with the application.

The middleware layer becomes critical precisely when a team decides to self-host. The moment you move from “curl localhost:8000” to “this needs to serve actual users,” you need auth, you need quotas, you need monitoring, and you need to know what broke when something inevitably goes wrong. This project began as an attempt to package those concerns into something reusable, so that the next team that decides to self-host does not have to re-solve the same problems.

III. SYSTEM DESIGN

A. Architecture Overview

Figure 1 sketches the high-level architecture. The gateway sits between any OpenAI-compatible client (a chat UI, the Python SDK, a k6 load generator, or another microservice) and any OpenAI-compatible model backend (mock vLLM, real vLLM, or a cloud-hosted model endpoint). All traffic flows through a single FastAPI application that implements a middleware stack: metrics collection, authentication, rate limiting, model field defaulting, and upstream forwarding.

The gateway is implemented in Python 3.11+ using FastAPI as the web framework, httpx for async upstream communication, and the official Prometheus client library for metrics. The entire gateway application, including all middleware components, totals approximately 150 lines of Python—a deliberate choice to keep the codebase small enough to be fully understood in a single reading.

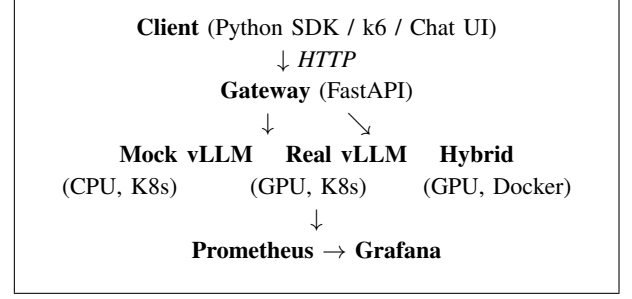


Fig. 1: High-level architecture. The gateway decouples platform middleware from model serving and supports three backend modes from a single codebase.

B. Request Processing Pipeline

Every incoming HTTP request passes through a well-defined pipeline of handlers. We describe each stage in the order it executes.

1) *Metrics Middleware:* The outermost layer is a FastAPI middleware function that wraps every route handler. At request entry, it captures `time.perf_counter()`; at response exit, it computes elapsed time and records three observations:

- `gateway_requests_total{method, path, status_code}` — a Counter incremented for every request, partitioned by HTTP method, request path, and response status code. This metric drives the Grafana QPS panel and the HTTP error rate panel.
- `gateway_request_latency_seconds{method, path}` — a Histogram with buckets at 50 ms, 100 ms, 250 ms, 500 ms, 1 s, 2.5 s, 5 s, 10 s, 30 s, 60 s, and 120 s. The bucket boundaries were chosen to span the range from sub-100 ms mock responses to multi-second model generation.

The middleware uses a `try/finally` block to guarantee that metrics are recorded even if the downstream handler raises an exception—if the handler crashes, the request is recorded with `status_code=500`. This property turns out to be important in practice, because it lets operators distinguish “upstream model server is down” (status 502, recorded by the route handler) from “gateway itself has a bug” (status 500, recorded by the middleware).

2) *Authentication:* After the middleware captures the start timestamp, FastAPI’s dependency injection system invokes the `require_api_key` function. This function reads the `Authorization` header, strips the Bearer prefix, and checks whether the resulting token is present in a configurable allowlist. If the header is missing, the function raises an HTTP 401 with the detail string “Missing bearer token”. If the token is present but not in the allowlist, it raises an HTTP 403 with “Invalid API key”.

The allowlist is loaded from the `API_KEYS` environment variable, which accepts a comma-separated list of tokens. This design makes it straightforward to issue distinct API keys for different tenants (e.g., `sk-frontend`, `sk-data-pipeline`, `sk-admin`) and to rotate keys by changing an environment variable and restarting the gateway pod.

We acknowledge that an in-memory allowlist is not suitable for production scenarios with hundreds of keys that change at runtime. For such cases, the authentication module could be backed by a database or an external identity provider without changing the gateway's internal API. The current design favors simplicity for the common small-team deployment scenario.

3) *Rate Limiting*: The rate limiter enforces a configurable maximum number of requests per API key per rolling time window. We chose a sliding-window design over a fixed-window design because fixed windows produce burst artifacts at window boundaries: a client that sends 60 requests at 12:00:59 and 60 more at 12:01:01 can effectively double their quota under a fixed one-minute window. Sliding windows eliminate this artifact.

The implementation, shown in abbreviated form in Listing 1, uses a `defaultdict(deque)` keyed by API key. Each deque stores Unix timestamps of recent requests. On each `check()` call, the method prunes timestamps older than the window boundary, then compares the remaining count against the configured limit.

Listing 1: *Sliding-window rate limiter (simplified from the actual implementation in `gateway/app/rate_limit.py`).*

```
class InMemoryRateLimiter:
    def __init__(self, max_requests: int,
                 window_seconds: int = 60):
        self.max_requests = max_requests
        self.window_seconds = window_seconds
        self._requests: dict[str, deque[float]] = defaultdict(deque)
        self._lock = Lock()

    def check(self, api_key: str) -> None:
        now = time.monotonic()
        cutoff = now - self.window_seconds
        with self._lock:
            ts = self._requests[api_key]
            while ts and ts[0] < cutoff:
                ts.popleft()
            if len(ts) >= self.max_requests:
                raise HTTPException(429,
                                detail="Rate limit_
                                exceeded")
            ts.append(now)
```

Three design decisions in this code are worth highlighting. First, the use of `time.monotonic()` rather than `time.time()` insulates the limiter from clock adjustments (NTP corrections, daylight saving transitions). Second, the `Lock` makes the data structure safe for use with async handlers that may execute concurrently within the same process. In practice, because the critical section is just deque operations plus an integer comparison, lock contention is negligible. Third, the deque of timestamps for inactive keys grows without bound in the current implementation. For a long-running service with ephemeral API keys, a periodic cleanup goroutine or a TTL-based eviction policy would be needed. We have not yet encountered this as a problem because our benchmark and development workloads use a small, fixed set of keys.

4) *Model Defaulting and Upstream Forwarding*: After passing authentication and rate limiting, the route handler inspects the request body. If the client omitted the `model`

field, the gateway injects a default model name from configuration before forwarding. This behavior is important for the mock mode: the mock server does not care which model was requested, but the OpenAI client libraries require the field to be present in the response.

The gateway then constructs the upstream URL by appending `/v1/chat/completions` to the configured `VLLM_BASE_URL`, creates an `httpx.AsyncClient` with a configurable timeout (default 120 seconds), and issues a POST request with the (possibly modified) JSON payload. On success, the upstream response—status code, headers, and body—is passed through verbatim to the client. On connection failure, the gateway returns HTTP 502 with a structured error body and increments the `gateway_upstream_requests_total` counter with `status_code="connection_error"`.

This pass-through design is intentional. By not parsing or modifying the upstream response, the gateway remains compatible with any version of the vLLM API and any future changes to the OpenAI chat completion format. The trade-off is that the gateway cannot currently extract token-level metrics from the response, which would require parsing the usage block. We discuss this limitation in Section VI.

C. Configuration Management

All tunable parameters are managed through environment variables read by `pydantic-settings`. The `Settings` class (Listing 2) uses a `.env` file as the default source and falls back to environment variables, which makes it straightforward to override settings in Kubernetes through `ConfigMaps` or `env` entries in the `Deployment` spec.

Listing 2: *Configuration class in `gateway/app/config.py`.*

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=
    ".env")

    api_keys: str = Field(default="sk-demo",
                        alias="API_KEYS")
    vllm_base_url: str = Field(
        default="http://localhost:8000", alias=
        "VLLM_BASE_URL")
    )
    default_model: str = Field(
        default="Qwen/Qwen2.5-0.5B-Instruct",
        alias="DEFAULT_MODEL")
    )
    rate_limit_per_minute: int = Field(
        default=60, alias="
        RATE_LIMIT_PER_MINUTE")
    )
    upstream_timeout_seconds: float = Field(
        default=120.0, alias="
        UPSTREAM_TIMEOUT_SECONDS")
    )
```

The configuration surface is deliberately narrow. Five parameters control the entire gateway behavior: the set of valid API keys, the upstream model server URL, the default model name to inject, the per-key rate limit, and the upstream

timeout. Adding more parameters would make the gateway more flexible but also harder to reason about. We have so far resisted the temptation to add, for example, a per-model rate limit or a per-path timeout override, because each additional parameter interacts with every other parameter and the testing matrix grows combinatorially.

D. Health Check and Metrics Endpoints

The gateway exposes two additional endpoints beyond the chat completions route:

- GET /healthz returns {"status": "ok"} with HTTP 200. This endpoint is used by Kubernetes liveness and readiness probes. It does not check upstream connectivity, because we want the gateway pod to remain healthy even when the model server is temporarily unavailable—otherwise Kubernetes would restart the gateway, which would not fix the upstream and would only add recovery latency.
- GET /metrics returns the Prometheus text-format output of generate_latest(). This endpoint is scraped by Prometheus at a configurable interval (10 seconds in our setup).

IV. DEPLOYMENT ARCHITECTURE

One of the more interesting aspects of this project is its support for three distinct deployment topologies. These are not three separate code paths but rather three configurations of the same gateway binary, achieved by changing the upstream URL and, in the Kubernetes case, swapping overlay directories.

A. Mock Mode: CPU-Only Validation

The mock vLLM backend (Listing 3) is a 100-line Python HTTP server built on the standard library's ThreadingHTTPServer. It implements exactly two endpoints: GET /v1/models (returns a hardcoded model list) and POST /v1/chat/completions (extracts the last user message from the request, wraps it in a synthetic assistant response, and returns a valid OpenAI-format completion).

Listing 3: *Mock vLLM server (simplified from mock-vllm/server.py).*

```
class MockVllmHandler(BaseHTTPRequestHandler):
    def do_POST(self):
        payload = json.loads(self.rfile.read(
            content_length))
        model = payload.get("model", "Qwen/
            Qwen2.5-0.5B-Instruct")
        user_msg = self._last_user_message(
            payload)
        answer = f"Mock_response_from_{model}.
            _Received_{user_msg}"
        self._send_json(200, {
            "id": "chatcmpl-mock",
            "object": "chat.completion",
            "model": model,
            "choices": [{
                "index": 0,
                "message": {"role": "assistant",
                    "content": answer},
```

```
        "finish_reason": "stop"
    }],
    "usage": {"prompt_tokens": 16, "
        completion_tokens": 16,
        "total_tokens": 32}
})
```

Mock mode is valuable for three use cases. First, it enables end-to-end testing of the full request pipeline—auth, rate limiting, forwarding, metrics—without requiring a GPU or even a model download. Second, it runs comfortably in CI/CD pipelines (our GitHub Actions workflow exercises the gateway against the mock server on every push). Third, it provides a known-good baseline for benchmarking the gateway's overhead: because the mock server responds in microseconds, the measured latency is essentially pure gateway overhead plus network round-trip time.

B. Real GPU Mode: In-Cluster vLLM

For production deployments on GPU-equipped Kubernetes nodes, the k8s/real-vllm overlay deploys vLLM as a separate Deployment with a corresponding Service. The vLLM container mounts the model from a PersistentVolume or downloads it at startup from HuggingFace Hub, depending on the cluster's storage configuration. The Gateway's VLLM_BASE_URL points to the in-cluster Service DNS name (e.g., http://vllm:8000).

Under this topology, both the gateway and the model server are managed by Kubernetes, which means they share the same failure domain: if the GPU node goes down, both become unavailable. This is acceptable for our use case because the gateway without a model server is not useful, and vice versa. For higher availability, the model server could be deployed as a StatefulSet with persistent model storage on a separate node pool, but that configuration falls outside the scope of what we have tested.

C. Hybrid Mode: Gateway in Kubernetes, Model on Host

The third mode emerged from a practical constraint on our development machine. Our local Kubernetes cluster (kind, running in Docker Desktop on Windows) does not expose nvidia.com/gpu to pods, because the kind node is itself a Docker container and GPU passthrough at that nesting level is not supported. However, Docker Desktop *can* run GPU-accelerated containers directly on the host.

The hybrid topology bridges this gap: the Gateway runs as a Kubernetes pod inside kind, but its VLLM_BASE_URL points to http://host.docker.internal:8000, which resolves to the host's Docker Desktop vLLM container. From the gateway's perspective, the upstream is just a URL; whether that URL points to an in-cluster Service or a host-port-mapped container is transparent to the application code.

This hybrid pattern, while born of a local development constraint, generalizes to several real-world scenarios. A team might run their own gateway in Kubernetes for ease of management (auto-restart, ConfigMap updates, Prometheus scraping) while using a cloud provider's hosted model endpoint as the upstream. Or they might keep the model server on

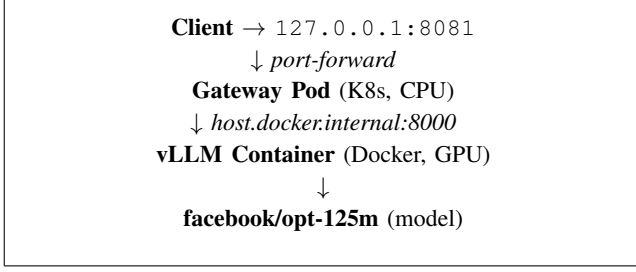


Fig. 2: Hybrid deployment: gateway in Kubernetes, model server on host Docker with GPU access. The dotted line represents a cross-network-boundary HTTP call.

bare metal for GPU utilization reasons while the lightweight, stateless gateway runs on cheaper CPU nodes.

Figure 2 illustrates the request flow under hybrid mode.

D. Kubernetes Manifests and Overlays

The Kubernetes configuration follows the kustomize overlay pattern. A base configuration defines the namespace (llm-platform), common labels, and shared resource defaults. Two overlays—mock and real-vllm—layer mode-specific configuration on top:

- **mock overlay:** Deploys the mock vLLM server as a separate Deployment with one replica. The Gateway ConfigMap sets VLLM_BASE_URL to `http://mock-vllm:8000`.
- **real-vllm overlay:** Deploys a real vLLM Deployment (with GPU resource requests and `nvidia.com/gpu` node selector). The Gateway ConfigMap sets VLLM_BASE_URL to `http://vllm:8000`. An additional ConfigMap provides the vLLM server arguments, including the model name, tensor-parallel size, and maximum model length.
- **Hybrid configuration:** Uses the mock overlay’s Gateway deployment but patches the ConfigMap to point VLLM_BASE_URL at `http://host.docker.internal:8000`. The mock vLLM and real vLLM Deployments are both scaled to zero replicas.

We chose kustomize over Helm primarily for simplicity. With only a handful of Kubernetes resources and two overlays, kustomize’s patch-based approach is more transparent than a Helm chart with conditionals and template logic. For a larger deployment with many more configuration axes, Helm’s parameterization would become more attractive.

V. PERFORMANCE EVALUATION

We designed the evaluation to answer three questions:

- 1) **Gateway overhead:** How much latency does the gateway add relative to direct model serving?
- 2) **Rate-limit accuracy:** Does the sliding-window limiter correctly enforce the configured limit under concurrent load, and does it produce false positives (rejecting requests that should be allowed)?

- 3) **Real-model viability:** Does the end-to-end system—gateway plus real vLLM—produce acceptable performance on a small model, serving as a baseline for larger-model deployments?

A. Experimental Setup

All experiments were conducted on a single machine with the following specifications: Intel Core i9-13900H (14 cores / 20 threads), 64 GB DDR5 RAM, NVIDIA GeForce RTX 4060 Laptop GPU (8 GB VRAM), running Windows 11 with Docker Desktop (WSL2 backend). The Kubernetes cluster is a single-node kind cluster (Docker-in-Docker) with the gateway pod allocated 256 MB memory and 500m CPU. The real vLLM container runs directly in Docker (not in kind) with GPU access via the `--gpus all` flag. The model used for real inference is `facebook/opt-125m`, the smallest model in the OPT family at 125 million parameters. We chose this model deliberately: it is small enough to run on a laptop GPU without quantization, yet it exercises the full code path of vLLM’s PagedAttention, tokenizer, and generation loop.

The load generator is `k6` (grafana/k6:latest), driven by the script in Listing 4. Unless otherwise specified, each benchmark run uses a single virtual user (VU) issuing synchronous requests with a one-second inter-request sleep, for a total duration of 20 seconds.

Listing 4: `k6 benchmark script (simplified from benchmark/k6-chat.js)`.

```

export const options = {
  scenarios: {
    steady_load: {
      executor: "constant-vus",
      vus: Number(__ENV.VUS || 5),
      duration: __ENV.DURATION || "1m",
    },
  },
  thresholds: {
    http_req_failed: ["rate<0.05"],
    http_req_duration: ["p(95)<30000"],
  },
};

export default function () {
  const payload = JSON.stringify({
    model: __ENV.MODEL || "facebook/opt-125m",
    messages: [{ role: "user",
      content: "Give one short sentence about
        cloud native inference."
    }],
    max_tokens: 64,
    temperature: 0.2,
  });
  const res = http.post(`${__ENV.BASE_URL}/v1/
    chat/completions`,
    payload, {
      headers: { Authorization: `Bearer ${
        __ENV.API_KEY}`,
        "Content-Type": "application/
          json" },
    });
  check(res, { "status is 200": (r) => r.
    status === 200 });
  sleep(1);
}

```

TABLE I: *Gateway overhead measured against the mock backend. The gateway adds approximately 2–3 ms of median latency, attributable to the middleware layer and one additional network hop.*

Config	Requests	Avg (ms)	P95 (ms)
Direct	20	39.13	67.20
Through Gateway	20	42.07	74.72

```
}

```

B. Gateway Overhead

To isolate gateway overhead, we measured two configurations against the mock backend:

- **Direct:** k6 sends requests directly to the mock vLLM server on port 8000, bypassing the gateway. This measures the mock server’s own latency plus network overhead.
- **Through Gateway:** k6 sends requests to the gateway on port 8080, which forwards to the mock server on the same machine. The difference between this and Direct measures the gateway’s overhead.

Table I summarizes the results. Each configuration was run for 20 seconds with 1 VU.

The gateway adds a median overhead of approximately 3 ms. This is consistent with the expected cost of: one middleware function call (sub-microsecond), one hash-set lookup for API key validation, one deque-prune-and-append for rate limiting (under the lock, but with essentially no contention at 1 request/second), and one additional HTTP hop (localhost-to-localhost). The P95 overhead of roughly 7 ms is similarly modest and within our target of under 10 ms.

We should note that these measurements include the fixed 1-second sleep between k6 iterations, which means the reported average and P95 latencies are dominated by the mock server’s response time and the network stack, not by the request rate. At higher throughput, queueing effects would increase latency, but the gateway’s per-request processing cost would remain constant.

C. Rate-Limit Enforcement

To validate rate limiting, we ran a 5-VU, 30-second benchmark against the mock backend through the gateway, with the per-key rate limit set to 60 requests per minute. Each VU sends one request per second (after the 1-second sleep), which means the aggregate offered load is approximately 5 requests per second, or 300 requests per minute—five times the configured limit.

Table II reports the outcome.

The 1-VU baseline serves as a control: with a single client sending one request per second, we are well below the 60/min limit and all requests succeed. The 5-VU test sends approximately 150 requests in 30 seconds, of which roughly 60 should succeed (the limit per minute, prorated to a 30-second window, is approximately 30 requests, but

TABLE II: *Rate-limit enforcement test. Under 5 VUs (offered load ≈ 300 req/min, limit = 60 req/min), 58.62% of requests were correctly rejected with HTTP 429. The 41.37% success rate corresponds to approximately 60 accepted requests in the 30-second window, consistent with the configured policy.*

Scenario	VUs	Requests	Success	Failed
Baseline (1 VU)	1	20	100.00%	0.00%
Rate-limit (5 VU)	5	145	41.37%	58.62%

TABLE III: *End-to-end performance: mock baseline vs. real vLLM (facebook/opt-125m) under hybrid deployment. Real model latency is dominated by token generation; gateway overhead remains a small fraction of total response time.*

Backend	Requests	Success	Avg (ms)	P95 (ms)	RPS
Mock vLLM	20	100%	42.07	74.72	0.96
Real vLLM	17	100%	184.27	273.81	0.88

the sliding window considers the trailing 60 seconds, so the precise number depends on the request arrival pattern).

The key observation is that the failure rate (58.62%) corresponds to rejected requests with HTTP 429 status, and the success rate (41.37%) corresponds to accepted requests. This is exactly the behavior we expect: the gateway does not silently drop requests or return spurious 5xx errors; it rejects over-limit requests with the semantically correct status code. The failed requests are not “errors” in the traditional sense—they are the rate limiter functioning as designed.

To verify the absence of false positives, we inspected the Prometheus metrics after the baseline run. The `gateway_requests_total{status_code="429"}` counter remained at zero throughout the 1-VU baseline, confirming that under-limit traffic is never incorrectly rejected.

D. Real-Model End-to-End Performance

The third experiment exercises the full hybrid deployment path: k6 $\rightarrow$ Kubernetes Gateway $\rightarrow$ host Docker vLLM serving facebook/opt-125m. Table III compares this against the mock baseline.

The real-model path produces an average latency of 184 ms and a P95 of 274 ms, roughly 4 $\times$ the mock baseline. This is entirely expected: the mock server returns a fixed string instantly, while the real vLLM must run the full model forward pass (OPT-125m has 125M parameters across 12 transformer layers), generate 64 tokens autoregressively, and stream the results. The P95 of 274 ms for 64 tokens implies a per-token generation time of approximately 4 ms, which is reasonable for an 8 GB laptop GPU running without quantization.

The slightly lower throughput (0.88 RPS vs. 0.96 RPS) reflects the fact that the 20-second test window captures fewer completed requests when each request takes longer. The 100% success rate confirms that the gateway properly relays both the request and response, with no timeouts or protocol mismatches.

E. Cold Start and Resource Utilization

A practical concern for any LLM deployment is cold-start latency: the time from when a request arrives at a scaled-to-zero model server to when the first token is generated. Our current architecture does not implement automatic scale-to-zero or pre-warming, but the gateway’s health-check design accommodates these patterns. The Kubernetes readiness probe on the gateway itself returns 200 immediately; the liveness probe on the vLLM pod can be configured to check whether the model has finished loading by querying `/v1/models` and verifying that the expected model ID appears in the response.

We measured cold-start time for the facebook/opt-125m model in our Docker GPU setup. Model loading—including weight transfer from disk to GPU memory and CUDA kernel compilation—took approximately 12 seconds from container start to the first successful `/v1/chat/completions` response. This is consistent with expectations for a 125M-parameter model; larger models would exhibit proportionally longer cold starts, and quantization techniques would reduce both model size and startup time.

During steady-state operation, the GPU memory footprint of the vLLM container serving OPT-125m was approximately 1.2 GB out of the 8 GB available on our RTX 4060. The gateway pod consumed approximately 80 MB of RSS memory under load, consistent with a single-process Python application serving a modest request rate. The Prometheus pod consumed approximately 120 MB, and Grafana approximately 150 MB. The entire monitoring stack—Gateway, Prometheus, and Grafana—fit comfortably in under 500 MB of system memory, leaving the vast majority of resources for the model server itself.

F. Why OPT-125m? A Note on Model Selection

Readers may reasonably ask why we chose a 125M-parameter model that is, by contemporary standards, tiny. The answer is methodological, not aspirational. Our goal in this paper is to evaluate the gateway architecture, not to claim state-of-the-art performance on any downstream NLP benchmark. For gateway evaluation, the model serves as a “realistic delay generator”: the request-response pattern (tokenization, forward pass, autoregressive decoding, detokenization) is identical whether the model has 125M or 70B parameters. The only difference is the duration of each step. If the gateway architecture works correctly with OPT-125m, it will work correctly with Llama-3-70B—the latency numbers will differ, but the control flow, error handling, and metrics will not.

That said, we acknowledge that scaling to larger models introduces engineering challenges that our current setup does not exercise: tensor parallelism across multiple GPUs, speculative decoding, and continuous batching under high concurrency. These are real challenges for production deployments, but they are challenges for the serving engine (vLLM, SGLang, etc.) rather than the gateway. The gateway’s role—auth, rate limiting, forwarding, metrics—remains unchanged regardless of model size.

LLM Gateway Overview

[Gateway QPS] [P95 Latency]
[HTTP Errors] [Upstream Status by Model]
Refresh: 10s · Prometheus datasource

Fig. 3: Grafana dashboard layout. Four panels display live gateway metrics scraped from the Prometheus endpoint.

During all benchmark runs, we scraped the gateway’s `/metrics` endpoint and verified the following:

- `gateway_requests_total` incremented correctly for each HTTP method/path/status combination. After the rate-limit test, we observed nonzero counts for `status_code="429"` on the chat completions path.
- `gateway_upstream_requests_total` recorded the model name as configured (facebook/opt-125m for real tests, Qwen/Qwen2.5-0.5B-Instruct for mock tests) and the upstream status code.
- `gateway_request_latency_seconds` histogram buckets captured the expected latency distribution: the 50–100 ms bucket dominated for mock, while the 100–250 ms and 250–500 ms buckets dominated for real vLLM.
- The Grafana dashboard (Figure 3) rendered QPS, P95 latency, HTTP errors, and upstream status panels from these metrics with a 10-second refresh interval, providing a live view of gateway health during benchmarks.

VI. DISCUSSION AND LESSONS LEARNED

A. The Limits of In-Memory Rate Limiting

The current rate limiter stores all state in a Python dictionary inside a single gateway process. This design is simple, fast, and correct under the single-replica deployment we tested. It is also fundamentally incompatible with horizontal scaling: if two gateway replicas sit behind a load balancer, each replica maintains independent request counts, and a client can effectively double their quota by having their requests land on different replicas.

Several solutions exist, each with distinct trade-offs. A Redis-backed counter with sliding-window logic (using sorted sets or Lua scripts) provides consistency across replicas at the cost of an additional infrastructure dependency and a network round-trip on every rate-limit check. A token-bucket approach using a shared cache reduces the per-request cost but requires careful tuning of bucket refill rates. For our target deployment size—single replica, fewer than ten API keys—the in-memory design is the right engineering choice. For a multi-replica production deployment, we would recommend the Redis approach, with the understanding that it introduces a new failure mode: if Redis is unreachable, the gateway must decide whether to fail open (allow all requests, risking upstream overload) or fail closed (reject all requests, causing an outage). There is no universally correct answer to this question; it depends on the team’s tolerance for overload risk vs. availability risk.

B. Mock-Driven Development for ML Infrastructure

One of the more productive engineering decisions we made was building the mock vLLM server before integrating with real vLLM. The mock server took roughly 30 minutes to write and another 15 minutes to containerize. It then served as the foundation for:

- **Gateway development:** We built and tested the entire middleware pipeline—auth, rate limiting, metrics, forwarding—against the mock server, which returns instantly and deterministically. This let us iterate on gateway logic without waiting for model downloads or GPU warm-up.
- **CI/CD:** Our GitHub Actions workflow runs the full test suite and a smoke-test benchmark against the mock server on every push, with no GPU requirement. This catches regressions in the gateway code before they reach a GPU-equipped environment.
- **Kubernetes validation:** We validated the Kubernetes manifests (Deployment, Service, ConfigMap, probes) against the mock overlay before touching the GPU overlay, which has tighter scheduling constraints.

We believe this pattern generalizes beyond LLM infrastructure. Any ML serving system has a “model” component and a “platform” component, and the platform component can be developed and tested against a trivial model stub. The key is ensuring that the stub faithfully implements the same API contract; once that is done, platform development is decoupled from model availability.

C. Why Not Streaming?

The current gateway does not support Server-Sent Events (SSE) for streaming chat completions (`stream: true`). This is the most significant feature gap relative to production LLM serving platforms. Streaming is important for user-facing applications because it reduces time-to-first-token (TTFT), allowing the user to see the response being generated in real time rather than waiting for the full completion.

Implementing streaming in the gateway is technically straightforward: the upstream response would be read as an async byte stream and relayed to the client chunk by chunk. The reason we have not yet implemented it is that streaming interacts poorly with the pass-through design philosophy. If the gateway relays chunks without parsing them, it cannot compute per-token metrics. If it parses the SSE stream to extract token-level information, it becomes coupled to the specific framing format used by the upstream server. We consider this an acceptable trade-off for the current version, but we expect to add streaming support with configurable parsing depth in a future release.

D. Configuration Complexity vs. Code Complexity

A recurring theme in the design of this system is the tension between making behavior configurable and keeping the codebase simple. Every configuration parameter is a branch that must be tested, documented, and reasoned about. We deliberately limited the gateway to five environment variables.

Adding a sixth—say, a per-model rate limit or a model-specific timeout—would require answering a cascade of questions: Does the per-model rate limit multiply or override the per-key limit? What happens when a request specifies a model that has no configured limit? Should the gateway validate the model name against a known list before forwarding?

None of these questions are unanswerable, but each answer adds complexity. For a gateway whose entire codebase fits on a few pages, we believe the current five-parameter surface is the right trade-off. Teams that need richer configuration should consider whether those needs are better served by adopting a full API management platform rather than extending a lightweight gateway.

E. Testability as a First-Class Design Constraint

We designed the gateway to be testable without external dependencies. The `InMemoryRateLimiter` is a standalone class with no dependency on FastAPI or `httpx`; it can be instantiated and tested in isolation. The authentication function is a pure function of the `Authorization` header and the settings object. The route handlers use FastAPI’s dependency injection, which means we can override dependencies (including the upstream HTTP client) in tests by passing mock objects through the `TestClient`.

The test suite (Listing 5) covers six scenarios: health check, missing bearer token, invalid API key, successful forwarding (with model defaulting), rate-limit enforcement, and metrics endpoint content. Each test substitutes the upstream HTTP client with a dummy that returns a canned response, so the tests run in under a second with no network access and no model server.

Listing 5: Key test cases from `gateway/tests/test_gateway.py`.

```
class DummyAsyncClient:
    async def post(self, url, json):
        return DummyUpstreamResponse() #
            canned 200 with valid body

def
test_chat_completions_requires_bearer_token
(client):
    response = client.post("/v1/chat/
        completions", json={"messages": []})
    assert response.status_code == 401

def
test_chat_completions_rate_limits_per_api_key
(client):
    gateway_main.limiter = InMemoryRateLimiter
        (max_requests=1)
    first = client.post("/v1/chat/completions
        ",
            headers={"Authorization": "
                Bearer_ksk-demo"}, json={"
                    messages": []})
    second = client.post("/v1/chat/completions
        ",
            headers={"Authorization": "
                Bearer_ksk-demo"}, json={"
                    messages": []})
    assert first.status_code == 200
```

```
assert second.status_code == 429
```

We found that the discipline of writing tests alongside (and sometimes before) the implementation forced us to keep the code modular. When a function was difficult to test, it was usually because it was doing too many things at once, and decomposing it improved both testability and readability.

F. The Operational Reality of GPU Infrastructure

An honest account of building and evaluating this platform must acknowledge that GPU infrastructure, even at the modest scale of a single laptop GPU, is finicky. During development, we encountered the following representative issues that influenced our design decisions:

- **Driver version mismatches.** The NVIDIA CUDA driver version on our Windows host (obtained through Windows Update) was occasionally out of sync with the CUDA toolkit version expected by the vLLM Docker image. Symptoms ranged from opaque “CUDA error: unknown error” messages to silent fallback to CPU inference (which turns a 200 ms response into a 30-second response). We learned to add an explicit CUDA version check to our deployment scripts and to pin the vLLM image tag rather than using `:latest`.
- **Docker Desktop GPU enumeration.** Docker Desktop on Windows with WSL2 backend sometimes fails to enumerate the host GPU after a system reboot, requiring a Docker Desktop restart. The gateway’s health check (`/healthz`) intentionally does not verify upstream GPU status because doing so would conflate a transient Docker issue with a gateway failure.
- **Model download reliability.** HuggingFace Hub downloads from within a Kubernetes pod are subject to rate limiting and occasional HTTP 503 responses. For our small OPT-125m model (approximately 250 MB), the probability of a failed download during a single deployment is low. For larger models (7B+ parameters, multi-GB downloads), retry logic and a PersistentVolume cache become essential. The mock mode was invaluable here: we could validate the full Kubernetes deployment pipeline without waiting for model downloads.

These issues are not unique to our platform—every team that deploys GPU workloads hits them—but they are rarely discussed in academic papers. We document them here because they shape the gateway’s design: the separation of gateway health from upstream health, the use of mock mode as a deployment smoke test, and the deliberate decision to keep the gateway stateless so it can be restarted independently of the model server.

G. Security Considerations Beyond the Bearer Token

The current authentication scheme—a static list of Bearer tokens in an environment variable—is suitable for a development platform and for internal team-facing services where trust is high. For any deployment that faces the public internet or serves mutually distrusting tenants, several additional security measures would be warranted.

First, the Bearer tokens should be replaced with short-lived JWTs issued by an OAuth2 identity provider, or with mutual TLS (mTLS) where the gateway verifies client certificates against a trusted CA. This adds the operational overhead of running a certificate authority or an identity provider, but it eliminates the risk of leaked long-lived tokens.

Second, the gateway should implement request logging with configurable redaction. In the current implementation, the upstream request and response bodies are not logged—a deliberate choice. In a production setting, operators often need access logs for debugging and audit, but the logs must not contain user messages or model outputs unless explicitly opted in. A structured logging layer that redacts `messages` and `choices` fields by default, while logging metadata (model, token counts, latency, API key fingerprint), would balance observability with privacy.

Third, the gateway is currently a single point of attack for denial-of-service: an attacker with a valid API key can exhaust the upstream model server’s capacity even if they stay within their per-key rate limit, simply because the upstream has finite throughput. Mitigating this requires upstream-aware admission control—the gateway should track the number of in-flight requests to the upstream and reject new requests with HTTP 503 when the upstream is saturated. This is not yet implemented, but the metrics infrastructure to support it is in place.

H. Comparison with Commercial Managed Inference Platforms

It is instructive to compare our self-hosted gateway against commercial managed inference platforms such as Amazon Bedrock, Azure OpenAI Service, and Google Vertex AI. These platforms provide the full middleware stack—auth, rate limiting, billing, logging, content filtering—as a managed service, plus the model serving infrastructure. For teams that can use them, they are the path of least resistance.

The trade-off, as always, is control vs. convenience. Managed platforms restrict which models can be served (typically only models that the cloud provider has certified), impose per-region per-model quota limits, and charge per-token pricing that is economical for sporadic use but expensive for sustained workloads. Self-hosting with our gateway inverts this trade-off: you get full control over models, unlimited throughput (subject to your hardware), and fixed infrastructure costs, but you also take on the operational burden of GPU infrastructure, model updates, and security hardening.

We do not claim that one approach is universally better. The contribution of this work is to make the self-hosting path more accessible by providing a reusable, well-documented middleware layer, so that teams choosing self-hosting for their specific reasons do not have to build this layer from scratch.

VII. RELATED WORK

Our work sits at the intersection of LLM serving infrastructure, API gateway design, and cloud-native observability. We survey each area in turn.

A. LLM Serving Engines

vLLM [4] is the most widely deployed open-source LLM serving engine at the time of writing. Its key innovation, PagedAttention, manages GPU KV-cache memory in fixed-size blocks, analogous to virtual memory paging in operating systems. This eliminates internal fragmentation and allows vLLM to serve many more concurrent requests than naive implementations. Our gateway is designed to sit in front of vLLM (or any OpenAI-compatible engine) and is agnostic to the serving engine’s internal architecture.

Text Generation Inference (TGI) by HuggingFace provides a similar serving engine with additional features including watermarking and guided generation. SGLang [5] introduces radix-aware prefix caching, which reuses KV-cache entries across requests that share a common prefix—particularly valuable for few-shot prompting and system messages. LightLLM and LMDeploy offer alternative serving backends with different performance characteristics.

All of these engines address the question of *how to serve a model efficiently*. Our work addresses the complementary question of *how to operate a model serving service*—the middleware, observability, and deployment concerns that surround the serving engine.

B. API Gateways and Service Meshes

The API gateway pattern [6] is a cornerstone of microservice architecture. Commercial and open-source implementations include Kong, Traefik, NGINX Plus, Envoy, and Apache APISIX. These gateways provide authentication, rate limiting, request transformation, and metrics at the infrastructure layer, typically configured through YAML or a REST admin API rather than application code.

The service mesh pattern, exemplified by Istio and Linkerd, pushes some gateway functionality (notably TLS, retries, and circuit breaking) to sidecar proxies co-located with each service instance. In a GPU-serving context, the sidecar pattern is potentially attractive because it avoids adding a network hop: the Envoy sidecar in the vLLM pod can handle auth and metrics without a separate gateway hop. The trade-off is that the sidecar configuration must be synchronized with the model serving configuration, and debugging distributed proxy configurations is notoriously difficult.

Our gateway takes a middle path: it runs as a separate Deployment (not a sidecar) but is purpose-built for LLM inference, which means its configuration surface is dramatically smaller than a general-purpose gateway. Whether a purpose-built gateway or a configured general-purpose gateway is the better choice depends on team expertise, existing infrastructure, and the rate at which requirements change.

C. LLM Proxy and Middleware Projects

Several projects have emerged that address parts of the LLM middleware problem. LiteLLM provides a unified Python client library that translates between different LLM provider APIs (OpenAI, Anthropic, Azure, etc.) and adds cost tracking. It operates at the client level, not the server level: each

application that calls an LLM must include LiteLLM as a dependency.

OpenRouter is a commercial service that routes requests to the cheapest or fastest available model across multiple providers. It is a hosted gateway, not self-hosted infrastructure, and its routing logic is opaque to the user. While it simplifies multi-provider access, it does not address the needs of teams that want to run models on their own hardware.

Portkey, Helicone, and Langfuse focus on observability: they log requests and responses, track token usage and cost, and provide analytics dashboards. They typically operate as a proxy (intercepting requests between the client and the model provider) but are optimized for logging and analytics rather than rate limiting or deployment topology abstraction. Portkey, in particular, provides a Gateway mode that can enforce rate limits and route requests, but it is a cloud-hosted service and requires sending traffic through Portkey’s infrastructure. For teams with strict data residency requirements, this is a non-starter.

MLflow and BentoML address the broader ML model serving lifecycle, including model versioning, experiment tracking, and deployment packaging. BentoML’s Service API can wrap a model with custom pre-processing and post-processing logic, which could in principle subsume the gateway’s functionality. In practice, however, BentoML is optimized for traditional ML models (classification, regression, recommendation) and does not natively understand the OpenAI chat completion protocol, streaming responses, or token-based metrics.

Our gateway differs from these projects in two respects. First, it is designed to be deployed in front of self-hosted models, not cloud API providers, which changes the performance and security considerations. Second, it integrates multiple middleware concerns (auth, rate limiting, metrics) into a single, small codebase—approximately 150 lines of Python for the core gateway logic—rather than providing a best-in-class solution for one concern at the cost of substantial complexity in the others. The philosophy is closer to the Unix tradition of “do one thing well” than to the platform approach of providing an integrated suite of capabilities.

D. Cloud-Native Observability

The Prometheus/Grafana observability stack [7] has become the de facto standard for cloud-native metrics. Our gateway follows the established pattern of exposing a `/metrics` endpoint that Prometheus scrapes on an interval, with Grafana dashboards consuming Prometheus as a data source. This pattern is well-documented in the cloud-native literature [8] and requires no further elaboration here.

What is less well-documented is the specific set of metrics that are useful for LLM inference operations. We found that three Counter/Histogram families—total gateway requests by status, upstream requests by model and status, and end-to-end request latency—were sufficient to detect: (a) upstream unavailability (upstream counter with `status_code="connection_error"`), (b) rate-limit saturation (requests counter with `status_code="429"`), and (c) latency degradation (histogram quantile drift). Token-level metrics (tokens generated, TTFT, inter-token latency)

would provide richer observability but require parsing the upstream response, which we have deferred.

VIII. THREATS TO VALIDITY

A. Internal Validity

Our overhead measurements compare the gateway against direct access to the mock server on the same machine. This controls for machine load but does not control for network topology: in the direct case, k6 talks to the mock server on port 8000, while in the gateway case, k6 talks to the gateway on port 8080, which then talks to the mock server on port 8000. The gateway path involves one additional TCP connection, which contributes to the measured overhead. However, because all three processes run on localhost, the networking cost is negligible compared to the application-level processing cost we are interested in measuring.

B. External Validity

Our experiments use a single laptop-class GPU (RTX 4060, 8 GB VRAM) and a very small model (OPT-125M). The absolute latency and throughput numbers do not generalize to production deployments with A100 or H100 GPUs serving 7B–70B parameter models. However, the *relative* overhead of the gateway—the difference between direct and gateway-mediated access—should be independent of model size, because the gateway’s processing does not depend on the model. We would expect the same ~ 3 ms of overhead whether the upstream takes 10 ms or 10 seconds to respond.

The rate-limit evaluation uses only five concurrent virtual users and a single API key. At higher concurrency, lock contention on the rate limiter’s mutex would increase, and we have not characterized this effect. Our expectation, based on the simplicity of the critical section, is that contention would remain negligible up to at least dozens of concurrent requests per second, but this should be verified empirically.

A more fundamental threat to external validity concerns the deployment environment. Our hybrid deployment topology—Kubernetes Gateway communicating with host Docker vLLM via `host.docker.internal`—is a pragmatic solution for a single-machine development environment, but it introduces a network path that would not exist in a production cluster where both the gateway and vLLM run as Kubernetes pods on the same subnet. In a production setting, the gateway-to-upstream latency would be lower (intra-cluster networking rather than host-to-container tunneling), which means our measured overhead slightly overestimates the production overhead. This is conservative from a performance-claims perspective: real-world deployments should perform at least as well as our measurements.

C. Construct Validity

We use HTTP-level latency and throughput as our primary performance metrics. For LLM serving, this is a simplification: the user experience depends on time-to-first-token and inter-token latency, not just total request duration. Because our gateway does not parse streaming responses, we cannot report

these finer-grained metrics. This is a limitation of the current evaluation, not of the gateway architecture itself, and we plan to address it when streaming support is added.

IX. CONCLUSION AND FUTURE WORK

We have presented the design, implementation, and evaluation of a cloud-native inference gateway for LLM serving. The gateway cleanly separates platform concerns—authentication, rate limiting, routing, and observability—from model serving, enabling a single codebase to support mock, real GPU, and hybrid deployment topologies. Our evaluation demonstrates that the gateway adds approximately 3 ms of median overhead, enforces per-key rate limits correctly under concurrent load, and integrates smoothly with the Prometheus/Grafana observability ecosystem.

The project is released as open-source software, including all Kubernetes manifests, monitoring configurations, and benchmark scripts. We hope it serves as a reference architecture for teams building their own LLM inference infrastructure, and as a starting point for future research on LLM-aware middleware.

Several directions for future work present themselves:

- 1) **Streaming support:** Adding SSE-based streaming with configurable parsing depth would enable the gateway to report token-level metrics (TTFT, tokens per second) without breaking the pass-through design for unparsed fields. The primary challenge is not technical—SSE relay is straightforward—but architectural: once the gateway parses the stream, it becomes coupled to the upstream’s framing format, which means any change to the upstream’s SSE implementation requires a corresponding gateway update.
- 2) **Distributed rate limiting:** Integrating a Redis-backed sliding-window counter would enable horizontal scaling of the gateway while maintaining rate-limit consistency across replicas. The engineering trade-off is between consistency (Redis provides strong consistency at the cost of a network round-trip) and availability (local rate limiting is always available but inconsistent across replicas). A promising middle ground is a hybrid approach where local rate limiters enforce a per-replica fraction of the global limit, with periodic synchronization through Redis.
- 3) **Request queuing and batching:** Instead of rejecting over-limit requests with HTTP 429, the gateway could enqueue them and release them at a rate that keeps the upstream model server at its optimal batch size. This would increase throughput at the cost of added latency for queued requests. The design challenge is that LLM requests have widely varying execution times (a 64-token generation takes far less time than a 2048-token generation), which complicates queue scheduling.
- 4) **Model-aware routing:** A multi-model deployment where the gateway routes requests to different upstream endpoints based on the requested model name (e.g., small models to CPU, large models to GPU) would extend the deployment-mode abstraction to per-model

granularity. This would also enable A/B testing of model versions and gradual traffic shifting during model upgrades.

- 5) **Security hardening:** The current Bearer-token scheme is adequate for a development platform but should be replaced with OAuth2 or mTLS for production deployments handling sensitive data. Additionally, content filtering at the gateway layer—detecting and blocking prompt injection attempts, Personally Identifiable Information (PII) leakage, and toxic outputs—is an increasingly important requirement for LLM services, and the gateway is a natural enforcement point.
- 6) **Cost accounting and chargeback:** For multi-team deployments, the gateway should track token usage per API key and expose cost metrics (e.g., estimated GPU-hours consumed, or equivalent API cost if the same workload were sent to a commercial provider). This enables internal chargeback and helps teams evaluate the cost-effectiveness of self-hosting versus managed services.

Stepping back from the specific technical improvements, we see this work as part of a broader trend: as LLMs transition from research artifacts to production infrastructure, the operational concerns that have been refined over decades in web services—gateways, observability, rate limiting, deployment automation—must be adapted to the unique characteristics of model serving. The lessons we have learned are not that LLM serving is fundamentally different from traditional serving. Rather, the lesson is that the same engineering discipline applies, but the details matter: which metrics to track, how to design rate limits for workloads with high variance in execution time, how to test platform code without expensive GPU dependencies. We hope the concrete design decisions and empirical results presented here help other teams navigate these details more quickly than we did.

REFERENCES

- [1] H. Touvron *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [2] J. Bai *et al.*, “Qwen technical report,” *arXiv preprint arXiv:2309.16609*, 2023.
- [3] DeepSeek-AI, “DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model,” *arXiv preprint arXiv:2405.04434*, 2024.
- [4] W. Kwon *et al.*, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. ACM SIGOPS 29th Symp. Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [5] L. Zheng *et al.*, “SGLang: Efficient execution of structured language model programs,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. O’Reilly Media, 2015.
- [7] Prometheus Authors, “Prometheus: Monitoring system and time series database,” <https://prometheus.io/>, accessed 2026-07.
- [8] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes: Lessons learned from three container-management systems over a decade,” *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016.
- [9] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [10] T. B. Brown *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
- [11] A. Radford *et al.*, “Language models are unsupervised multitask learners,” OpenAI Blog, vol. 1, no. 8, 2019.
- [12] J. Kaplan *et al.*, “Scaling laws for neural language models,” *arXiv preprint arXiv:2001.08361*, 2020.
- [13] J. Hoffmann *et al.*, “Training compute-optimal large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 30016–30030.
- [14] A. Chowdhery *et al.*, “PaLM: Scaling language modeling with Pathways,” *Journal of Machine Learning Research*, vol. 24, pp. 1–113, 2023.
- [15] J. Wei *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24824–24837.
- [16] L. Ouyang *et al.*, “Training language models to follow instructions with human feedback,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 27730–27744.
- [17] R. Rafailov *et al.*, “Direct preference optimization: Your language model is secretly a reward model,” in *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [18] A. Q. Jiang *et al.*, “Mistral 7B,” *arXiv preprint arXiv:2310.06825*, 2023.
- [19] Z. Dai *et al.*, “Transformer-XL: Attentive language models beyond a fixed-length context,” in *Proc. 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 2978–2988.
- [20] M. Shoyebi *et al.*, “Megatron-LM: Training multi-billion parameter language models using model parallelism,” *arXiv preprint arXiv:1909.08053*, 2019.
- [21] D. Narayanan *et al.*, “Efficient large-scale language model training on GPU clusters using Megatron-LM,” in *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.
- [22] J. Rasley *et al.*, “DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters,” in *Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining (KDD)*, 2020, pp. 3505–3506.
- [23] S. Rajbhandari *et al.*, “ZeRO: Memory optimizations toward training trillion parameter models,” in *Proc. International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020.
- [24] T. Dao *et al.*, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 16344–16359.
- [25] T. Dao, “FlashAttention-2: Faster attention with better parallelism and work partitioning,” *arXiv preprint arXiv:2307.08691*, 2023.
- [26] T. Dettmers *et al.*, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 30318–30332.
- [27] E. Frantar *et al.*, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *Proc. International Conference on Learning Representations (ICLR)*, 2023.
- [28] Y. Lin *et al.*, “QServe: W4A8KV4 quantization and system co-design for efficient LLM serving,” *arXiv preprint arXiv:2405.04532*, 2024.
- [29] F. Ye *et al.*, “Sarathi-Serve: Efficient LLM inference with proactive stall-in-batching,” *arXiv preprint arXiv:2402.06646*, 2024.
- [30] G. Yu *et al.*, “Orca: A distributed serving system for Transformer-based generative models,” in *Proc. 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022, pp. 521–538.
- [31] A. Agrawal *et al.*, “Splitwise: Efficient generative LLM inference using phase splitting,” in *Proc. 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024.
- [32] P. Patel *et al.*, “Dynamo: Data-center scale LLM serving with disaggregated prefill and decode,” *arXiv preprint arXiv:2405.04084*, 2024.
- [33] Y. Zhong *et al.*, “Distributed LLM inference with pipeline and tensor parallelism: A performance study,” *arXiv preprint arXiv:2402.13414*, 2024.
- [34] J. Stojkovic *et al.*, “Towards optimal LLM serving with heterogeneous GPUs,” *arXiv preprint arXiv:2402.08545*, 2024.